

Société de plomberie chauffagiste

L'objectif de ce TD est de valider l'utilisation des notions de Technologies Logicielles vu en cours (Python, Git, Test et Docker) et de démontrer votre capacité à organiser votre travail. Concernant Git, les critères d'évaluation sont : la création d'issues et de branches, le recours à des Pull Requests, pas de commit réalisés directement dans la branche principale, réalisation de commits atomiques.

Le thème de ce TD est une entreprise de plomberie chauffagiste spécialisée dans l'installation et la maintenance d'équipements (chaudières, radiateurs, pompes à chaleur). Vous devrez créer :

- un script python `seeder.py` permettant d'insérer des équipements, des installations et des zones dans une base de données ;
- un script python `heating.py` permettant de récupérer des informations et de réaliser des traitements sur la base de données.

Le premier traitement attendu consiste à parcourir la base de données et vérifier si les données associées aux équipements sont valides.

Consignes

Créez un dépôt **privé** nommé `TD_HEATING_SYSTEM` sur Github.

Ajoutez moi (SMoraisDev) au dépôt.

Publiez **régulièrement** votre code et ses modifications dans le dépôt.

Exercice 1 - Implémentation des classes et tests

Vous allez débiter le projet en créant les classes `Equipement`, `Installation` et `Zone`.

La classe `Equipement` :

- encapsule un type d'équipement, une année d'installation, une puissance (en kW) ;
- possède une méthode publique `is_valid` qui retourne `True` si les données sont valides, `False` sinon.

Les données sont valides si :

- le type est parmi : "Chaudiere", "Radiateur", "Pompe a chaleur", "Chauffe-eau" ;
- l'année d'installation est un entier supérieur à 1995 et différent de 2019 ;
- la puissance est un entier compris entre 1 et 50 kW.

La classe `Installation` encapsule : une surface en m² ; une liste d'équipements ; une liste de zones.

La classe `Zone` :

- encapsule un nom (ex : “Salon”, “Chambre”, “Salle de bain”) ; une surface en m² ; un équipement associé.

Les données associées aux installations et zones sont valides si :

- la surface de l’installation est un multiple de 10 et inférieure à 300 m² ;
- les zones couvrent toute la surface (somme des surfaces = surface totale) ;
- les zones ont chacune une surface > 0 ;
- chaque zone est associée à un équipement existant dans la liste des équipements de l’installation;

Notez qu’un équipement peut être utilisé dans plusieurs zones.

Exercice 2 - Utilisation de Docker pour peupler une base de donnée MongoDB avec un script Python (procédure interactive)

Ici, nous allons jouer avec deux images Docker qui devront pouvoir communiquer entre elles. Pour cela, nous allons utiliser Docker pour créer un réseau que nous nommerons `mynet`

```
docker network create mynet
```

Pour les utilisateurs de Windows, il est possible que cet appel ne fonctionne pas. Dans ce cas, passez par la commande

```
docker network create --driver nat mynet
```

Commencez par lancer un premier conteneur basé sur l’image `mongo` que vous nommerez `mongodb` et que vous connecterez au réseau `mynet` (utilisez l’option `--network mynet`).

Ensuite, lancez un second conteneur basé sur l’image `python:alpine` que vous nommerez `python-mongo` et que vous connecterez au réseau `mynet`. Afin d’éviter que le conteneur de l’image `python` ne se ferme automatiquement, il faudra utiliser le mode “foreground” (-t, -i ou -it).

Dans le conteneur `python-mongo` installez la paquet `pymongo` que nous allons utiliser pour remplir la base de donnée.

Utilisez l’interpréteur python du conteneur, pour peupler la base de données avec des informations associées équipements (type, année, puissance) en vous inspirant du code suivant :

```
import pymongo
from pymongo import MongoClient

client = MongoClient(host="A_REEMPLIR")
```

```

# Acces a la base de donnees "NOM_BASE_DE_DONNEES"
db = client["NOM_BASE_DE_DONNEES"]

# Acces a la collection "NOM_COLLECTION"
col = db["NOM_COLLECTION"]

# Ajout d'un 'fruit' dans la collection
fruit = {
    "nom": "banane",
    "couleur": "jaune"
}
res = col.insert_one(fruit)

# Verification de l'ajout
print(f'Le fruit {res.inserted_id} a bien ete cree')

# Localise et affiche le fruit
col.find_one()

```

En parallèle, vous pouvez vous assurer que la base de donnée à bien été remplie depuis le conteneur `mongo`. Pour cela, interagissez avec celui-ci en exécutant la commande `mongosh`. Une fois réalisé, utilisez la base de donnée que vous défini depuis le conteneur python. `use NOM_BASE_DE_DONNEES` Ensuite, interrogez la base pour récupérer un élément qui compose la collection que vous avez défini. `db.NOM_COLLECTION.findOne()`

Une fois cette étape réalisée, appelez l'enseignant pour qu'il constate que vous avez validé cet exercice !

Exercice 3 - Création d'un script Python peuplant la base de données et publication de l'image associée

Ecrivez un script Python `seeder.py` permettant de peupler la base de donnée. Pour cela, vous allez utiliser le code contenu dans les tests réalisés à l'Exercice 1 et ajouter des entrées valides et non valides. Pour commencer, vous pouvez uniquement peupler des équipements.

Une fois le script écrit, il ne vous reste plus qu'à définir le Dockerfile associé où l'on doit retrouver:

- l'installation des dépendances nécessaires
- la copie des données nécessaires
- l'exécution du script `seeder.py`

Maintenant, vous allez construire votre image en la taggant par `$DOCKERID/seed_mongo` (où `DOCKERID` est votre identifiant docker hub). Puis, après vous être connecté à Docker Hub, vous allez publier votre image.

Exercice 4 - Création d'un script Python traitant la base de donnée

Ecrivez un script Python `heating.py` qui doit se connecter à la base de données et regarder que les informations des équipements sont valides. Dans le cas où une donnée n'est pas valide, il doit l'afficher à l'écran.

Note : pour itérer sur une collection obtenus via `pymongo`, vous pouvez vous inspirer du snippet suivant:

```
fruits = col.find()
for fruit in fruits:
    nom = fruit["nom"]
    couleur = fruit["couleur"]
```

Exercice 5 - Assemblage des différents conteneurs avec Docker Compose !

Votre fichier `docker-compose.yml` va se composer de trois services :

- `mongodb` : que vous allez lancer à partir de l'image `mongo:latest`
- `seed_mongo` : que vous allez lancer à partir de votre image `$DOCKERID/seed_mongo` (ou de celle d'un de vos camarades) et qui dépendra de `mongodb`
- `heating` : que vous allez construire à partir des sources de l'Exercice 4 (il vous faudra faire un nouveau Dockerfile dédié à ce conteneur) et qui dépendra de `seed_mongo`

Note : dans le cas du conteneur `mongodb`, n'oubliez pas que le service `mongod` utilise le port 27017.

Pour vérifier que cela fonctionne vous pouvez exécuter la commande

```
docker-compose up --build
```

puis depuis un autre terminal exécuter la commande

```
docker exec -it NOM_DU_CONTENEUR bash
```

et enfin exécuter votre code python réalisant le traitement sur la base de donnée.

Exercice 6 - Bonus

Modifiez `heating.py` pour en faire une interface en ligne de commande (CLI). Pour commencer vous pouvez recouvrir vos développements existant, i.e. ajouter une option “`-check`” permettant de regarder si les informations sont valides.

Ensuite vous pouvez proposer d’autres options :

- rechercher les équipements d’un certain type, année d’installation, puissance inférieur à une valeur, ...
- supprimer les données invalides dans la base de données
- ...